



Ferrari



DES_Head-Unit Jo & Jackie

Vehicle App Develop Project with
Optimized OS based on RPi

Contents

1. Project Overview
2. System Architecture
3. Yocto Layers
4. Custom Layer
5. Head Unit App
6. Demo
7. Trouble Shooting

Project Overview

The **Head Unit project** aims to implement a **Yocto-based operating system optimized for vehicles** and to develop a **multimedia application** on top of it.



Why Yocto?

- alternative of RSPi OS -> setting for real ECU
- least packages



HU & IC

- Infotainment System
- communication between two apps

System Architecture

DES Automotive Cockpit System
Raspberry Pi 4B (ARM64 Cortex-A72 @ 1.5GHz)
Yocto Linux (Poky Scarthgap) + systemd init

Application Layer

HeadUnit APP

- Qt6 QML UI
- Wayland Client
- D-Bus Client
- HDMI-A-1

Instrument Cluster APP

- Qt6 QML UI
- Wayland Client
- D-Bus Client
- HDMI-A-2

PiRacer Controller

- USB Gamepad Input
- Throttle/Steering Control
- D-Bus com.des.vehicle
- CAN Bus Transmission
- Adafruit dependency

Middleware Layer

D-Bus System Bus

Interfaces:

- com.des.vehicle.Gear
- Policy:
- /etc/dbus-1/system.d/

Wayland Compositor

- VC4 KMS Driver
- Dual HDMI Output
- GPU Compositing
- XDG Shell Protocol
- DRM/KMS Backend

Connectivity Stack

- Connman (WiFi)
- WPA supplicant

Audio Stack

- Pulse Audio Server
- BlueZ-ALSA

Kernal&HAL Layer

SocketCAN Kernel Stack

- MCP2518FD SPI Driver
 - can0 Interface (500kbps)
 - SPI0 CS0/CS1
 - HDMI-A-1
- Module:
- i2c-dev

Linux Driver Tree

Overlays:

- vc4-kms-v3d
- Sseed-can-fd-hat-v2

Boot & Init

- U-Boot Bootloader
- Device Tree Overlays
- Plymouth Boot Splash
- Systemd-udev
- RPM Package Manager

Hardware Layer

RaspberryPi 4B

- BCM2711 SoC
- VideoCore VI GPU (128MB)
- 4GB RAM

CAN HAT

- MCP2518FD
- SPI BUS
- 12MHz Osc
- 500kbps

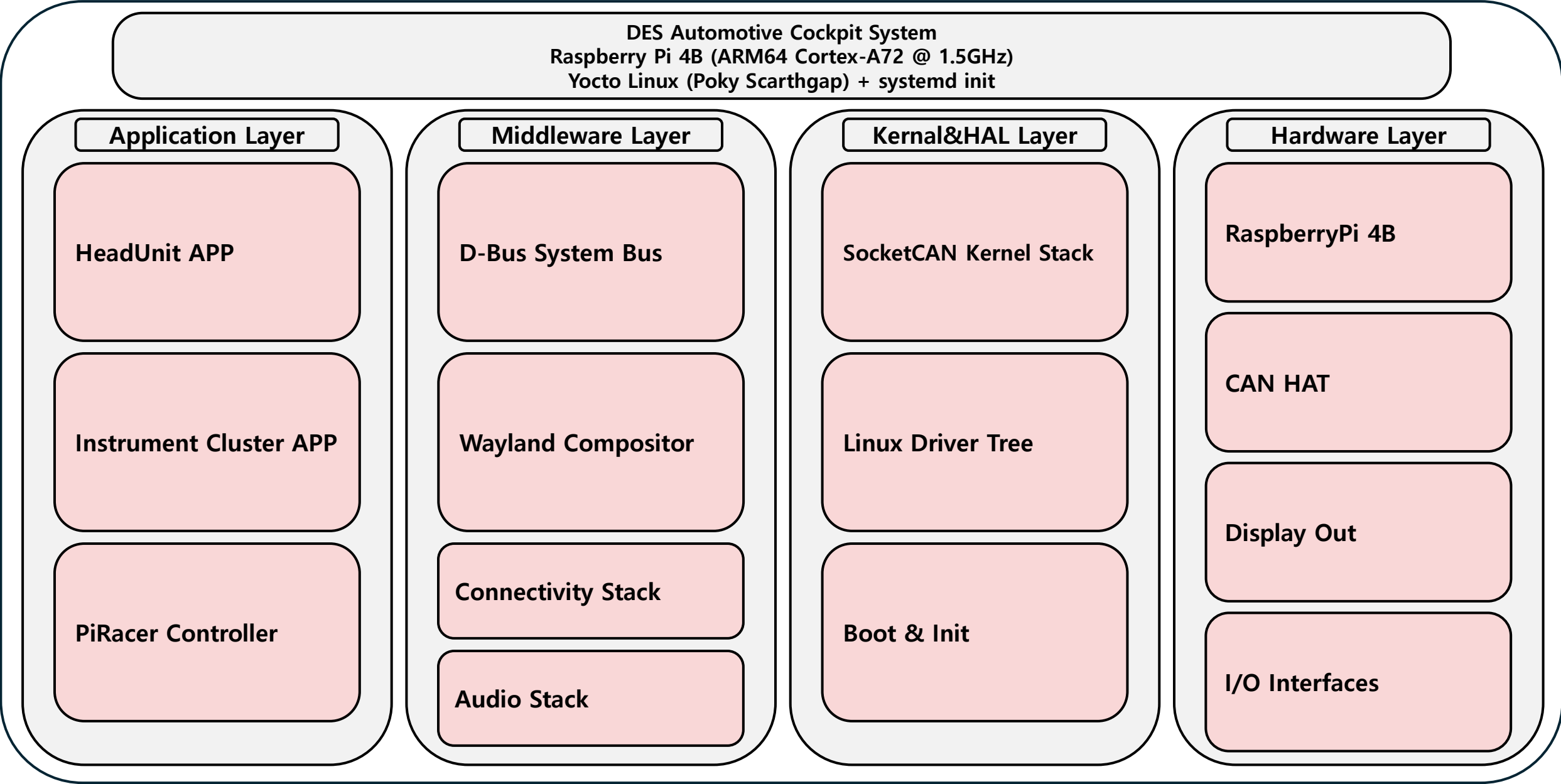
Display Out

- HDMI-A-1 (1024x600)
- HDMI-A-2 (1024x600)

I/O Interfaces

- USB 2.0/3.0
- GPIO 40-pin
- I2C Bus

System Architecture



Yocto Layers

meta-custom

meta-app

meta-env

meta-piracer

meta-qt6

meta-raspberrypi

meta-openembedded/poky

meta-custom

meta-custom

meta-app

- Head Unit
- Instrument Cluster

meta-env

- Graphic
- i2c
- Bluetooth
- Wifi
- Dbus

meta-piracer

- adafruit
- Piracer-Controller
- CAN
- Python Packages

meta-custom

meta-custom

meta-app

meta-env

meta-piracer

- Head Unit
- Instrument Cluster

meta-custom

meta-custom

meta-app

meta-env

meta-piracer

- Graphic
- i2c
- Bluetooth
- Wifi
- Dbus

meta-custom

meta-custom

meta-app

meta-env

meta-piracer

- adafruit
- Piracer-Controller
- CAN
- Python Packages

Head Unit Application



Head Unit Application

Head Unit app is the core of the in-vehicle infotainment system. Integrates various features. Provides intuitive user experience.

Ambient Light

- Implemented using Qt Design exclusively
- Adjustable colors and brightness based on user settings

Music Player

- Initial USB-based → Bluetooth integration
- Applied modern music playback methods
- Supports music streaming through smartphone connectivity

Gear Display

- Receives gear information from Instrument Cluster
- IPC through D-Bus
- Real-time gear status display and updates

Map Display

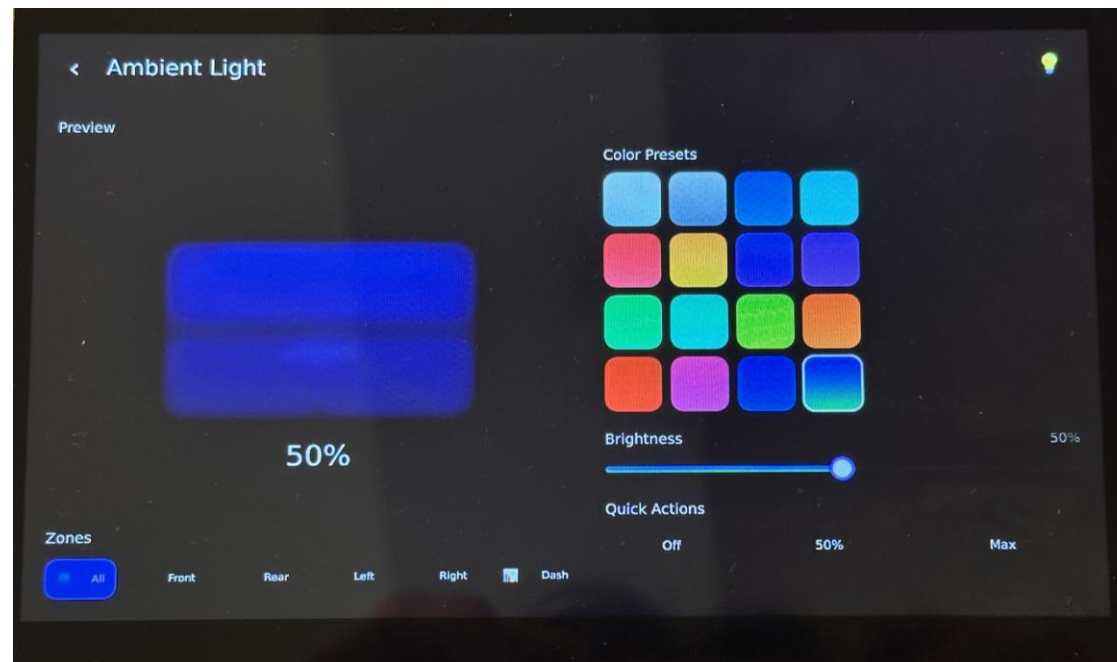
- Utilizing OpenStreetMap
- Setting point starts at current lat coordinates
- Focus on map display functionality without real-time GPS

Head Unit Application

Head Unit app is the core of the in-vehicle infotainment system. Integrates various features. Provides intuitive user experience.

Ambient Light

- Implemented using Qt Design exclusively
- Adjustable colors and brightness based on user settings

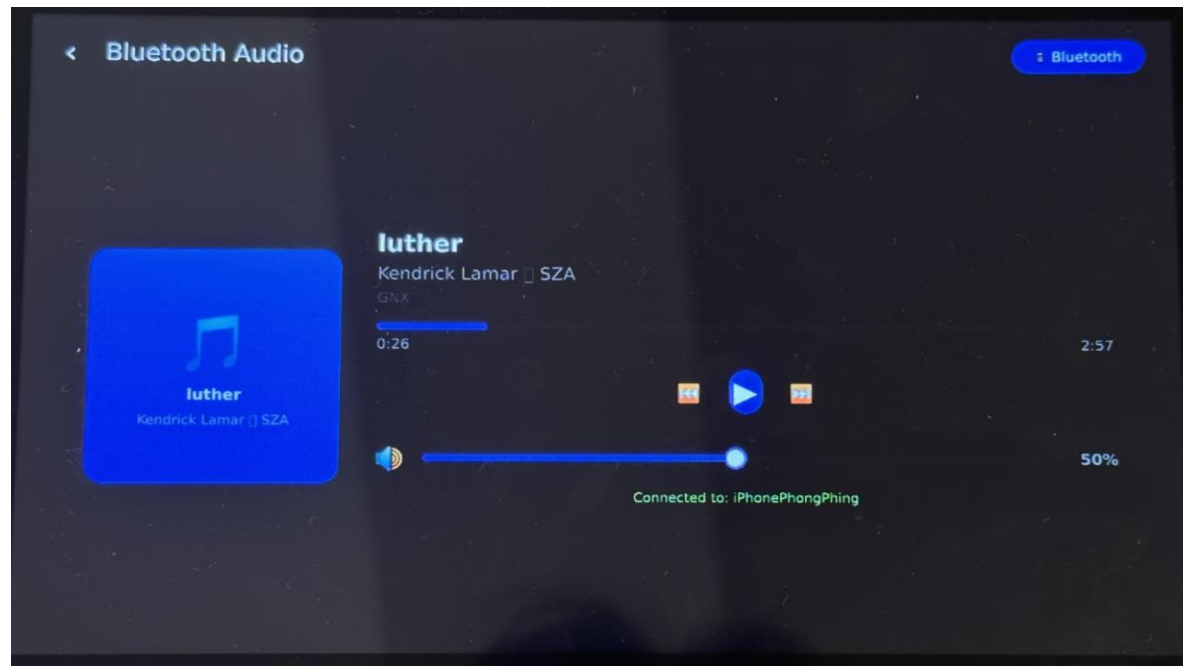


Head Unit Application

Head Unit app is the core of the in-vehicle infotainment system. Integrates various features. Provides intuitive user experience.

Music Player

- Initial USB-based → Bluetooth integration
- Applied modern music playback methods
- Supports music streaming through smartphone connectivity

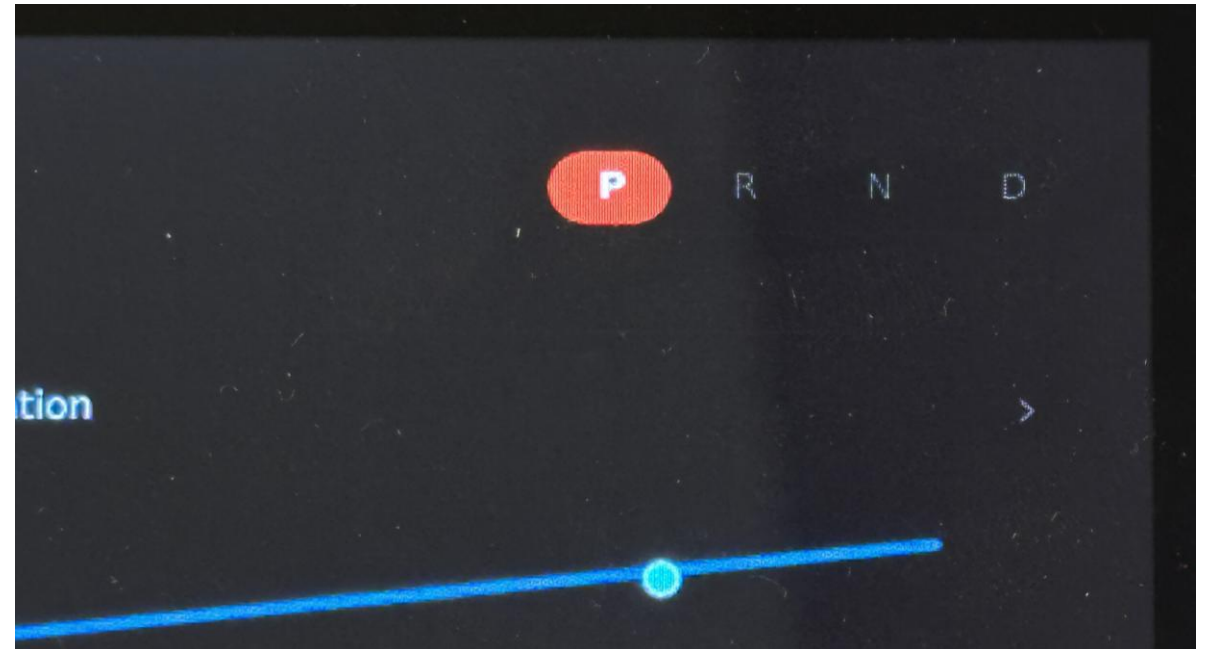
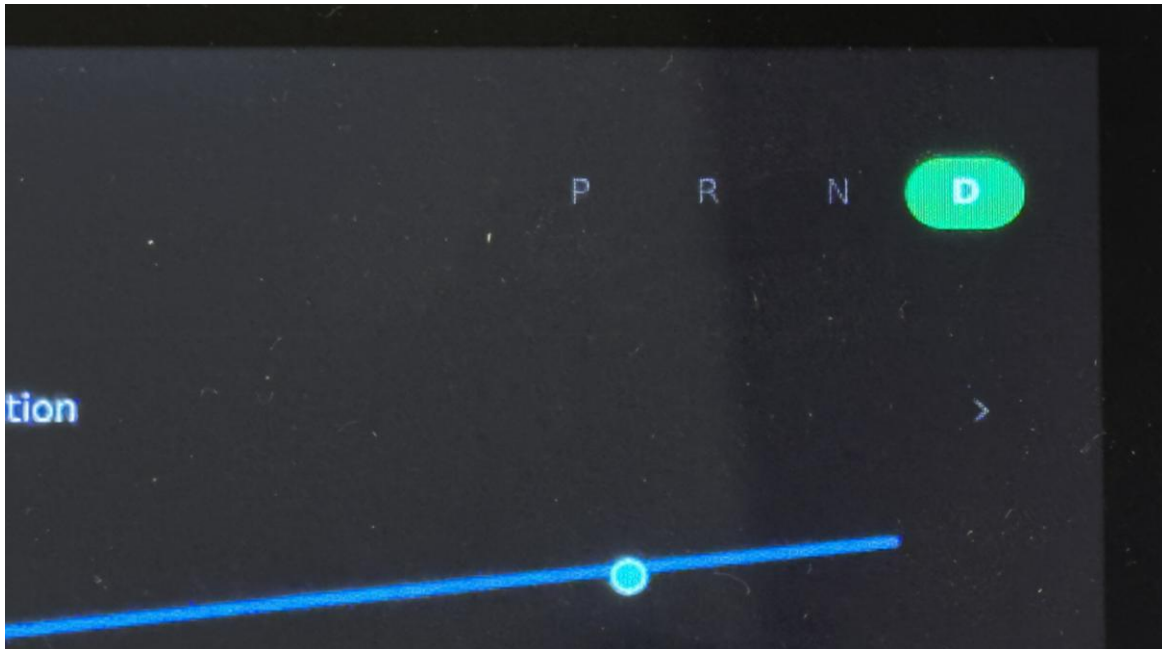


Head Unit Application

Head Unit app is the core of the in-vehicle infotainment system. Integrates various features. Provides intuitive user experience.

Gear Display

- **Receives gear information from Instrument Cluster**
- **IPC through D-Bus**
- **Real-time gear status display and updates**

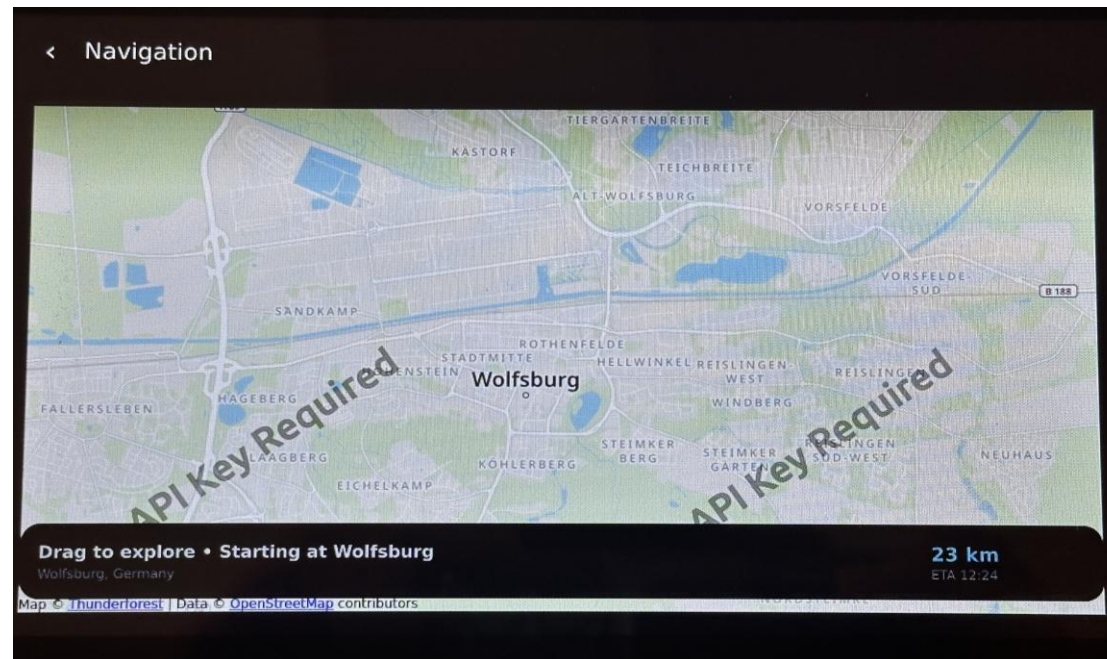


Head Unit Application

Head Unit app is the core of the in-vehicle infotainment system. Integrates various features. Provides intuitive user experience.

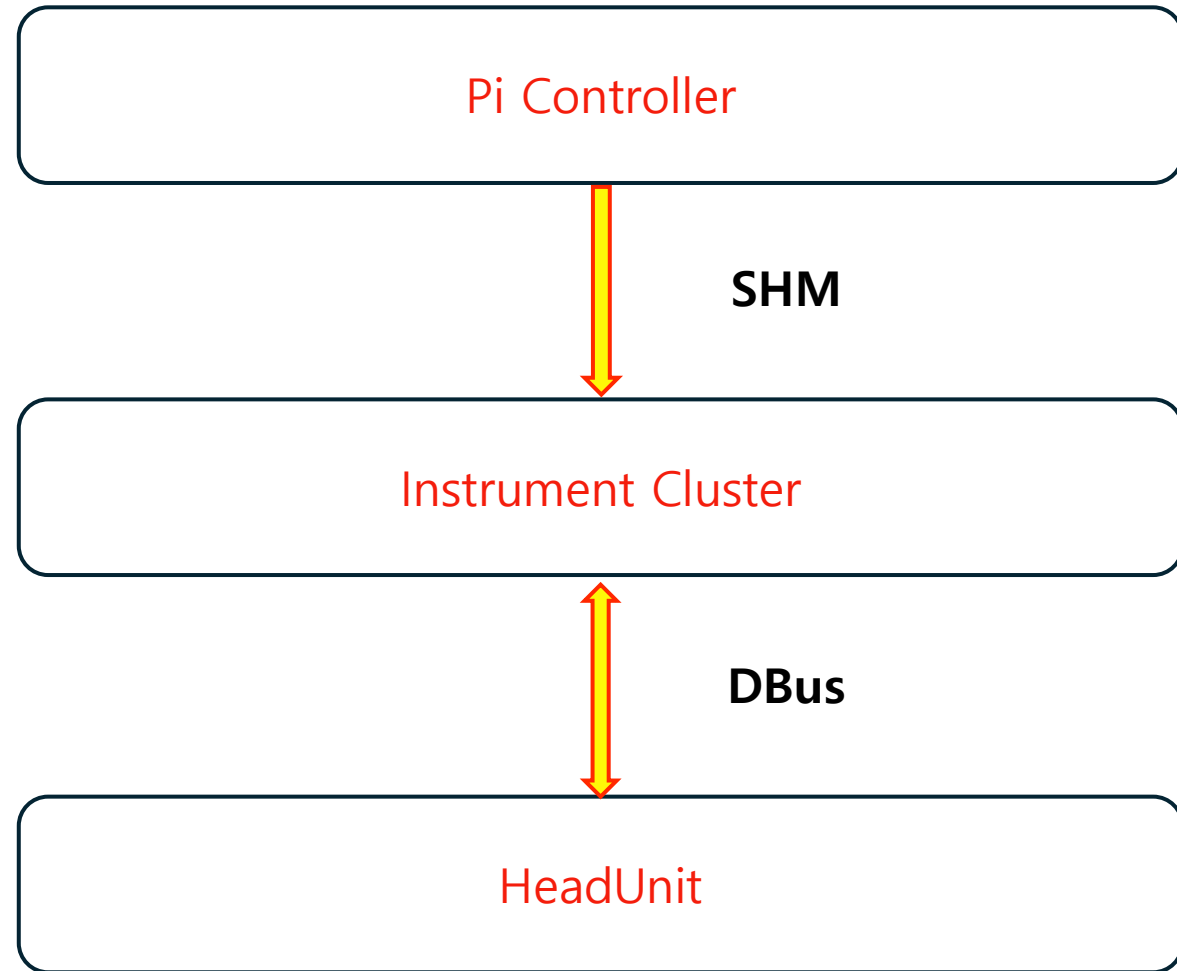
Map Display

- Utilizing OpenStreetMap
- Setting point starts at current lab coordinates
- Focus on map display functionality without real-time GPS



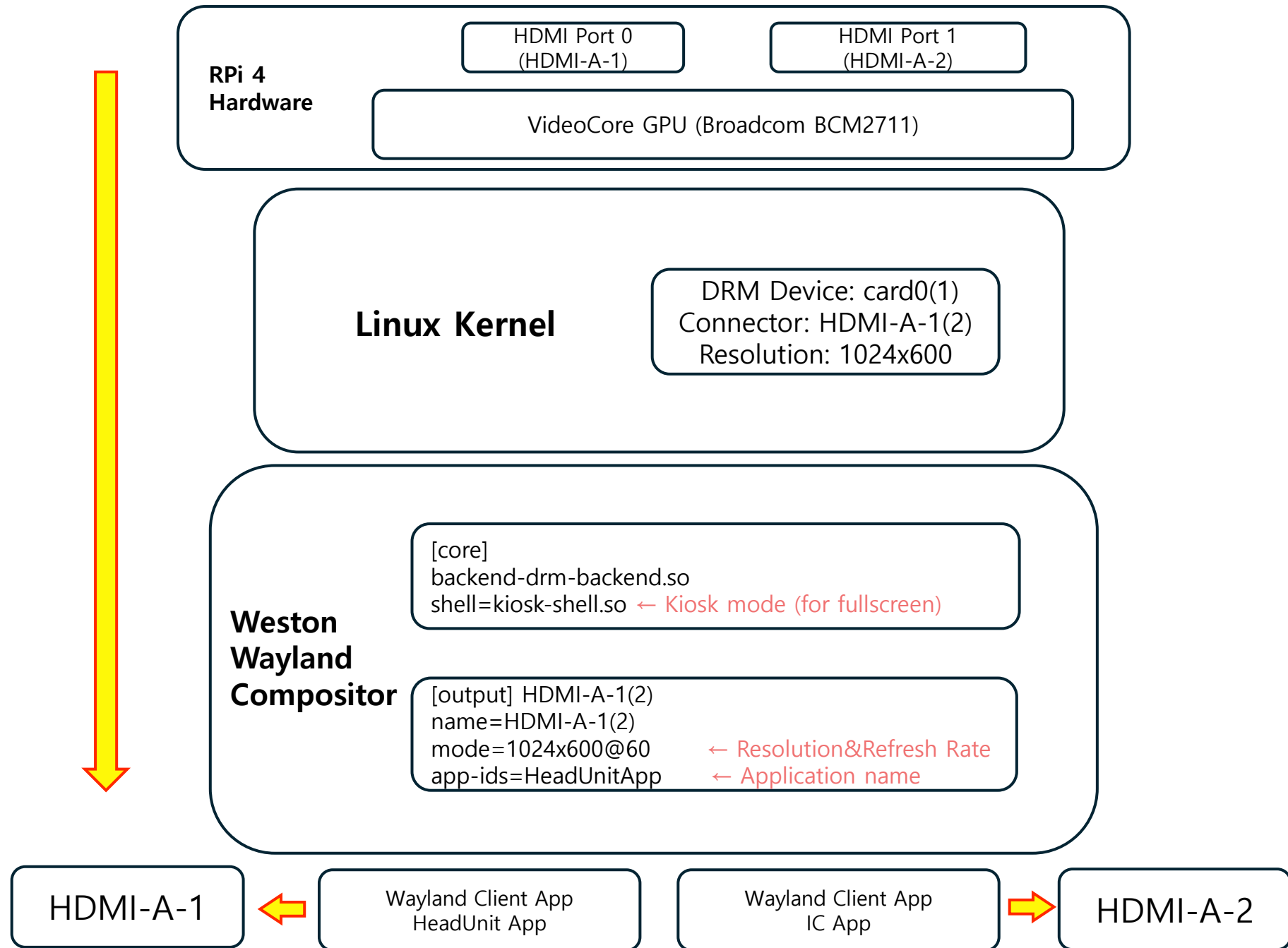
Head Unit Application

Communication



Head Unit Application

Weston Dual Monitor Architecture



Head Unit Application

Bluetooth Data Flow

Phase 1: Initializing & Broadcasting



Start App

1. Initialize BluetoothManager

- Create QBluetoothLocalDevice
- Register BluetoothAgent on D-Bus
 - Load saved device list

2. Start Broadcasting (User clicks the "Pair" button in the UI)

- startBroadcasting()
- Set "Discoverable" = true
- Set "Pairable" = true

3. Phone searches for device

- Phone Bluetooth settings →
"raspberrypi-64" appears

Phase 2: Pairing

Head Unit Application

Bluetooth Data Flow

Phase 2: Pairing



[User taps "raspberrypi-64" on the phone]

1. BlueZ invokes BluetoothAgent

D-Bus Method Call:

- RequestConfirmation(device, passkey)
- AuthorizeService(device, uuid)
- RequestPinCode(device)

2. BluetoothAgent handles auto-approval

3. QtBluetooth receives pairing event

- handlePairingFinished(address, Paired)
- Set device as "Trusted" via D-Bus
- stopBroadcasting() – no longer discoverable

Phase 3: Connection

Head Unit Application

Bluetooth Data Flow

Phase 3: Connection



Auto-connection attempt after pairing complete

1. Connection Check Timer (runs every 2 seconds)

- checkConnectionStatus()
- Scan all /org/bluez/hci0/dev_* devices

2. Connected Device Discovery

```
if device.Paired == true && device.Connected == true:  
    connectedDeviceAddress_ = device.Address  
    connectedDeviceName_ = device.Name
```

3. BluetoothAudioPlayer Integration

```
audioPlayer_ -> setConnectedDevice(devicePath)
```

Phase 4: Media Control

Head Unit Application

Bluetooth Data Flow

Phase 4: Media Control



Music playback from phone

1. BlueZ MediaPlayer1 sends PropertiesChanged signal

```
D-Bus Signal:
org.freedesktop.DBus.Properties.PropertiesChanged
├─ Interface: "org.bluez.MediaPlayer1"
├─ Changed Properties:
│   ├── "Status" = "playing"
│   ├── "Track" = {
│   │   ├── "Title": "Bohemian Rhapsody",
│   │   ├── "Artist": "Queen",
│   │   ├── "Album": "A Night at the Opera",
│   │   └── "Duration": 354000 (milliseconds)
│   └── }
└─ "Position" = 0
```

2. BluetoothAudioPlayer receives signal

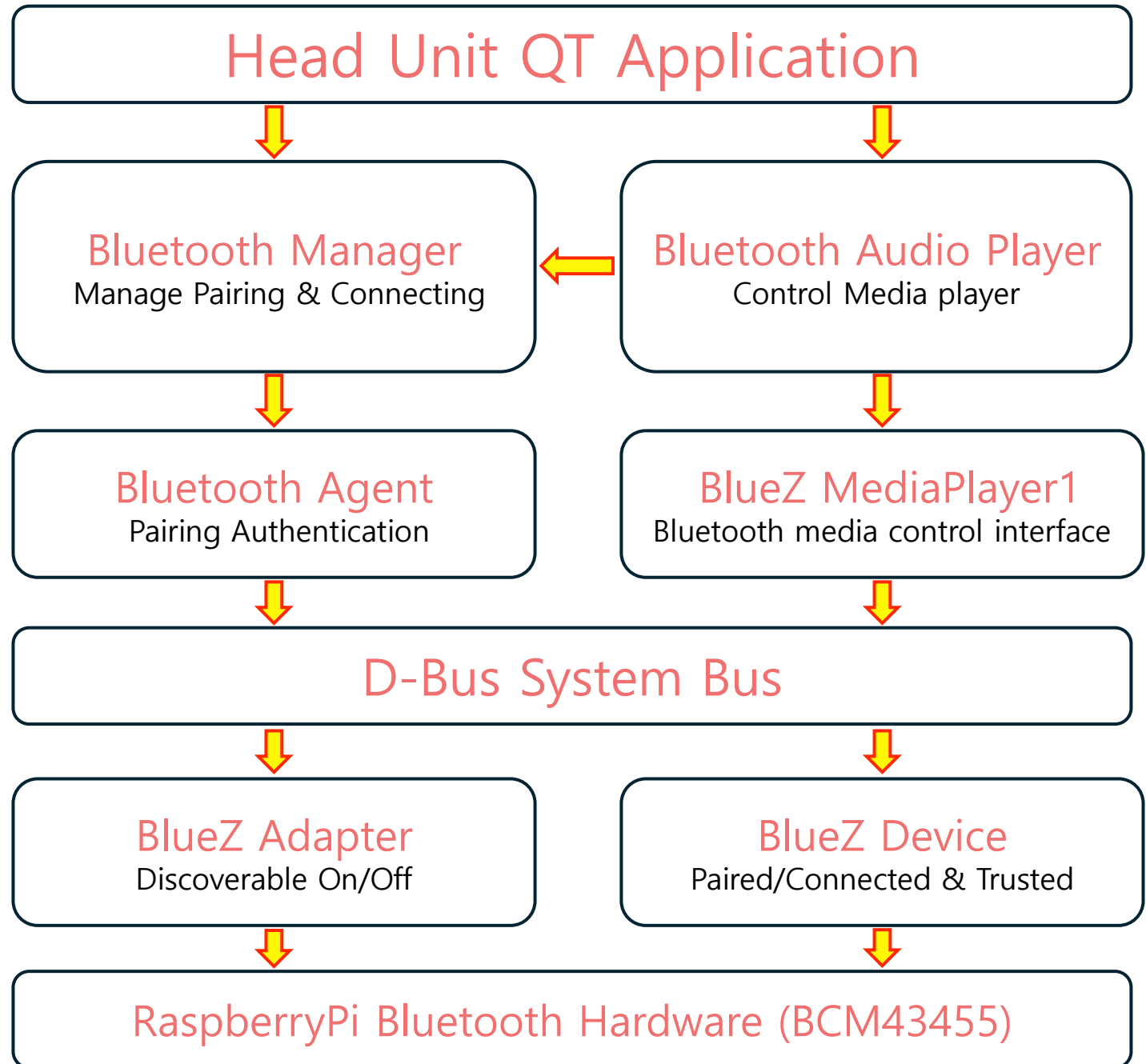
```
onMediaPlayerPropertiesChanged()
├─ updateMetadata(metadata)
│   ├── trackTitle_ = "Bohemian Rhapsody"
│   ├── trackArtist_ = "Queen"
│   ├── trackAlbum_ = "A Night at the Opera"
│   └── duration_ = 354000
├─ emit trackTitleChanged(), etc.
└─ downloadAlbumArt(artUrl) - Download album art
```

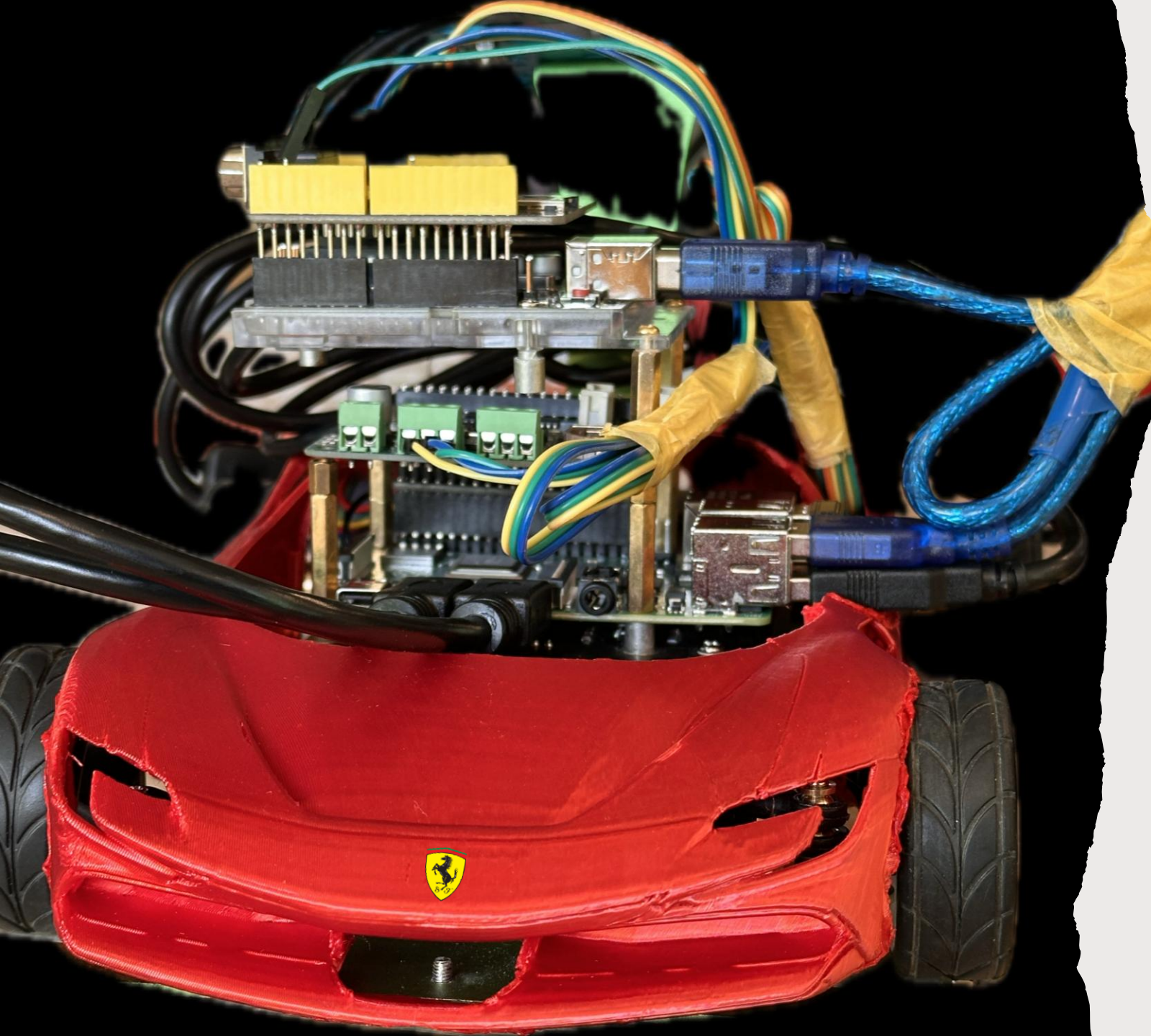
3. Qt Signal → QML UI Update

```
Display in QML's BluetoothScreen.qml:
├─ Text { text: btAudio.trackTitle }
├─ Text { text: btAudio.trackArtist }
├─ Image { source: btAudio.albumArtPath }
└─ ProgressBar { value: btAudio.position / duration }
```

Head Unit Application

Bluetooth System Architecture





Demo

Look at our JJ-90!!!

An aerial view of a Ferrari Formula 1 pit stop. A red Ferrari F1 car, number 55, is positioned in the center. It is surrounded by approximately 15 pit crew members in red racing suits and helmets. The scene is set on a grey asphalt track with yellow and white markings. The text "Trouble Shooting" is overlaid in white, bold, sans-serif font across the middle of the image.

Trouble Shooting

Trouble Shooting

with Yocto

- OOM when building -> swap file, memory limit, thread limit
- local.conf -> INHERIT += "rm_work"
- Display(Weston) –EGLFS couldn't display two Qt applications (HU, IC on two monitors)
 - **Solution:** Migrated to Wayland compositor (Weston) based architecture

Trouble Shooting

with Github CI/CD

- Github runner -> self-hosted runner
- Too much time every time I push -> full build / Qt build / app build splitting workflow

Trouble Shooting

• Notion for documenting and describing

Yocto 빌드 시도 일지 / 단계별 정리:

▼ Yocto 기본 이미지들 빌드해보기

- ▶ 기본적인 이미지를 만들기에 앞서, 아래 코드로 기본적인 패키지 설치:
(이미지 빌드할때 즉, 병 공장 돌리기 전에 필요한 원재료를 모은것임.)
- ▶ poky레이어부터 쌓기 시작
- ▶ 레이어 추가까지 완료했으면, local.conf 파일 수정한다음에 이미지 빌드하기
`. poky/oe-init-build-env build-des` 로 PATH에 경로추가하기
`bitbake des-image` 로 image 빌드하기
- ▶ [트러블슈팅](#)

▶ Yocto 기본 이미지들 빌드 후

▶ 부팅과정

▶ 네트워크/IP 확인

▶ CI/CD 구축 with Github Action

▶ 경로 합치기

▶ 새로 부팅해도 기존 아주 기본 qml윈도우 뜨는 일 발생

▶ 파일구조 정리하기

▶ 모니터 연결 문제 트러블슈팅

▶ 부팅 플로우

`LICENSE_FLAGS_ACCTITLE` append = "synaptics-killswitch"도 명시 수록.

위 조치 후 `bitbake des-image` 로 재빌드를 진행

• 디스크 부족

해서 파티션 확장도 해주었다.(200GB로 확장했음)
그럼에도 부족해서 안쓰는것들 정리하는중...

```
sudo du -sh /var/lib/snapd/snaps /var/lib/snapd/cache
sudo du -sh /var/lib/snapd | sort -h
df -h /
```

해당 코드로 안쓰는 snap revision들이 얼마나 차지하는지 보고,

```
#확인용: 삭제할 항목만 출력
snap list --all | awk '/disabled/ {print $1, $3}'
#각 disabled 버전 한번에 삭제 실행
sudo bash -c 'snap list --all | awk "/disabled/ {print \$1, \$3}" | while read nan
```

대형 PCH(`cmake_pch.hxx.gch`):

`tmp-glibc/work/.../build/.../cmake_pch.hxx.gch` 파일이 300~420 MB씩 여러 개 있음.
빌드를 계속하면 또 생기기 때문에, `rm_work`를 활성화해 자동으로 정리.

`build-des/conf/local.conf` 에 `INHERIT += "rm_work"`를 추가

→ 각 레시피가 패키징을 끝낸 후 자기 WORKDIR을 정리.

특정 레시피는 `RM_WORK_EXCLUDE = "qtdeclarative qtquick3d"`처럼 예외 처리 가능.

• 부트파티션 공간부족

`do_image_rpi_sdimg` 단계에서 /boot 파티션(기본 48 MiB) 용량 부족으로 mcopy가 실패.
`BOOT_SPACE`를 넉넉히 키웠음(`BOOT_SPACE = "98304"` → 약 96 MiB).

`des-image.bb`

GitHub QR Code

